Shruggie Markdown — Technical Specification

Attribute	Value
Project Name	Shruggie Markdown
Project Slug	`shruggie-md`
CLI Command	`shruggie-md`
Repository	github.com/shruggietech/shruggie-md
License	TBD
Version	Pre-release (0.1.0 target)
Author	William Thompson (Shruggie LLC, DBA ShruggieTech)
Latest Revision Date	2026-03-24
Document Status	DRAFT
Audience	AI-first, Human-second

Table of Contents

- 1. Document Information
  - 1.1. Purpose
  - 1.2. Scope
  - 1.3. Terminology
- 2. Project Overview
  - 2.1. Project Identity
  - 2.2. Design Goals
  - 2.3. Non-Goals
- 3. Architecture
  - 3.1. Technology Stack
  - 3.2. Build Targets
  - 3.3. Shared Core Architecture
- 4. Visual Design Language
  - 4.1. Design Philosophy

- 4.2. Design References
- 4.3. Color System
- 4.4. Typography
- 4.5. Spacing and Layout
- 4.6. Component Styling
- 4.7. Motion and Transitions
- 4.8. Iconography

- 5. Application Layout
  - 5.1. View Modes
  - 5.2. Full-View Mode
  - 5.3. Split-View Mode
  - 5.4. Library View
  - 5.5. Settings View

- 6. Core Features
  - 6.1. Viewing
  - 6.2. Editing
  - 6.3. Converting
  - 6.4. Remote URL Fetching

- 7. Editor Engine
  - 7.1. CodeMirror Integration
  - 7.2. Syntax Highlighting
  - 7.3. Markdown Linting

- 8. Markdown Rendering
  - 8.1. Rendering Pipeline
  - 8.2. Selectable Engines
  - 8.3. Rendered Content Styling

- 9. Library Mode
  - 9.1. Purpose
  - 9.2. Setup
  - 9.3. Library Table
  - 9.4. Traversal and Filtering
  - 9.5. Extension Rules

- 10. Configuration and Preferences
  - 10.1. Configuration Architecture

- 10.2. Theme and Appearance
- 10.3. Editor Preferences
- 10.4. File Extension Rules
- 10.5. Storage Locations

- 11. Platform Targets
  - 11.1. Desktop Application
  - 11.2. Chrome Extension
  - 11.3. Progressive Web App

- 12. Repository Structure
- 13. Future Considerations

---

1. Document Information

1.1. Purpose

This document is the authoritative technical specification for Shruggie Markdown, a cross-platform markdown viewer, editor, and converter. It is written for an AI-first, Human-second audience. Its primary consumers are AI implementation agents operating within isolated context windows. Every section provides sufficient detail for an AI agent to produce correct implementation decisions without requiring interactive clarification.

1.2. Scope

In Scope:

- Desktop application for Windows 11, macOS, and Linux via Tauri.
- Chrome extension for the Chrome Web Store.
- Progressive Web App (PWA) installable from the browser.
- Markdown viewing with live preview, theming, and syntax highlighting.
- Markdown editing with split-view, linting, and configurable engines.
- Markdown-to-HTML and markdown-to-PDF conversion.
- Library mode for flat file management on desktop installs.
- Configuration and preference persistence.

Out of Scope:

- Authenticated remote URL fetching (see §13).
- Collaborative or multi-user editing.
- Cloud sync or account systems.
- Mobile-native applications (Android/iOS).

1.3. Terminology

Term	Definition
Full-view	The default display mode showing only the rendered markdown preview.
Split-view	A side-by-side display mode showing the editor pane alongside the rendered preview pane.
Library	The flat file management interface available only in native desktop installs.
Engine	The markdown compilation library used to transform markdown source into HTML.
Linter	A static analysis tool that checks markdown source for style and correctness issues.
Mount	The act of configuring a local directory as the source for library mode.

2. Project Overview

2.1. Project Identity

Property	Value
Product name	Shruggie Markdown
Organization	ShruggieTech (Shruggie LLC)
Author	William Thompson
Repository	`github.com/shruggietech/shruggie-md`
CLI command	`shruggie-md`

Shruggie Markdown is the product name. It is what users install, what the documentation refers to, and what appears in the application title bar and store listings. **shruggie-md** is the project slug used for the repository name, CLI command, and package identifier.

2.2. Design Goals

- **G1 — Clean and intuitive.** The UI prioritizes clarity, whitespace, and quiet confidence over feature density. Every visible element earns its place.
- **G2 — Cross-platform from one codebase.** Desktop, Chrome extension, and PWA targets share a single frontend codebase with platform-specific shells.
- **G3 — Fast and responsive.** File opens, preview renders, and view toggles feel instantaneous. Heavy operations (PDF export, large file rendering) show non-blocking progress indicators.
- **G4 — Configurable without complexity.** Sensible defaults for casual users, deeper configuration available for power users. No setting requires documentation to understand.
- **G5 — Offline-capable.** The desktop application and PWA are fully functional without network access. The Chrome extension requires network access only when fetching remote URLs.

2.3. Non-Goals

Shruggie Markdown is explicitly not:

- A full-featured IDE or code editor.
- A note-taking application with organizational hierarchy (folders, tags, backlinks).
- A real-time collaboration tool.
- A CMS or publishing platform.

---

3. Architecture

3.1. Technology Stack

Layer	Technology	Purpose
Desktop shell	Tauri 2	Native window, filesystem access, CLI registration, system tray
Frontend framework	React 18+ with TypeScript	UI components, state management, rendering logic
Styling	Tailwind CSS 4	Utility-first styling with custom design tokens
Editor	CodeMirror 6	Markdown editing, syntax highlighting, linting integration
Markdown rendering	markdown-it (default)	Markdown-to-HTML compilation
PDF export	Browser print API with custom print stylesheet	HTML-to-PDF conversion pipeline
Build tooling	Vite	Frontend bundling for all targets
Chrome extension	Manifest V3	Chrome Web Store distribution
PWA	Vite PWA plugin + Workbox	Service worker, offline caching, install prompt

3.2. Build Targets

Target	Shell	Distribution
Windows 11 (x64)	Tauri	GitHub Releases (`.msi` installer)
macOS (arm64)	Tauri	GitHub Releases (`.dmg` image)
Linux (x64)	Tauri	GitHub Releases (`.AppImage`)
Chrome extension	Manifest V3	Chrome Web Store
PWA	Service worker	Hosted static site

3.3. Shared Core Architecture

The application is structured as a shared React frontend with platform-specific integration layers. The frontend contains all UI, rendering, editing, and conversion logic. Each build target wraps this frontend in a thin shell that provides platform capabilities (filesystem access on desktop, extension APIs in Chrome, service worker caching for PWA).

```
shared frontend (React + TypeScript)
    ├── UI components (viewer, editor, settings, library)
    ├── markdown engine abstraction
    ├── theme system
    └── configuration state management
        │
        ├── Tauri shell (desktop)
```

```
        │   ├── native filesystem access
        │   ├── CLI entry point
        │   ├── file watcher (live reload)
        │   └── system preferences integration
        │
        ├── Manifest V3 shell (Chrome extension)
        │   ├── content script injection
        │   ├── browser action popup
        │   └── chrome.storage API
        │
        └── PWA shell
            ├── service worker (Workbox)
            ├── web app manifest
            └── install prompt handling
```

Platform-specific code is isolated behind a thin abstraction layer (`platform.ts`) that exposes a uniform API for file I/O, preference storage, and capability detection. The frontend queries platform capabilities at startup and conditionally enables features (e.g., library mode is enabled only when the platform reports filesystem access).

---

4. Visual Design Language

4.1. Design Philosophy

Shruggie Markdown adopts a design philosophy rooted in restraint. The interface should feel like a calm, focused environment where the user's content is the centerpiece and the application chrome recedes. Controls appear when needed and stay out of the way otherwise. Visual hierarchy is established through spacing and typography rather than heavy borders, color blocks, or decorative elements.

The following principles govern all visual design decisions:

- **Content over chrome.** The rendered markdown preview occupies maximum available space. Toolbars, sidebars, and controls are minimal in height and visual weight.
- **Quiet surfaces.** Backgrounds are flat and muted. No gradients, no drop shadows on content areas, no textured backgrounds.
- **Purposeful contrast.** Color is used sparingly and intentionally: to indicate interactivity (buttons, links), to signal state changes (active tabs, hover states), and to differentiate the editor pane from the preview pane.
- **Typographic clarity.** Font choices, sizes, and line heights are optimized for sustained reading of both rendered markdown and raw source.

4.2. Design References

The following design systems inform the visual language. No single reference is copied wholesale; each contributes a specific quality.

Reference	Contribution
Apple Human Interface Guidelines	Spacing rhythm (8pt grid), control sizing, subtle depth cues via thin borders rather than shadows, respect for system-level appearance preferences.
Linear	Information density without clutter, monochromatic palette with a single accent color, keyboard-first interaction patterns, smooth state transitions.
Notion	Clean typographic rendering of structured content, comfortable reading column widths, restrained use of UI elements around content areas.
Obsidian	Markdown-native editing experience, comfortable split-view proportions, respect for power users through configurable engine and theme options.

4.3. Color System

The color system is built on CSS custom properties defined at the `:root` level and scoped by a `data-theme` attribute on the document element. Three modes are supported: light, dark, and system (which follows `prefers-color-scheme`). Dark mode is the default.

All colors are defined as semantic tokens, never as raw hex values in component styles. The token vocabulary is deliberately small to enforce visual consistency.

Surface tokens:

Token	Purpose	Dark Value	Light Value
`--color-bg-primary`	Main application background	`#1a1a1a`	`#ffffff`
`--color-bg-secondary`	Sidebar, settings panels	`#222222`	`#f5f5f7`
`--color-bg-tertiary`	Editor pane background	`#1e1e1e`	`#fafafa`
`--color-bg-hover`	Interactive element hover state	`#2a2a2a`	`#ebebeb`
`--color-bg-active`	Active/selected element	`#333333`	`#e0e0e0`

Border tokens:

Token	Purpose	Dark Value	Light Value
`--color-border-primary`	Panel dividers, input borders	`#333333`	`#e0e0e0`
`--color-border-subtle`	Faint separators, table rules	`#2a2a2a`	`#eeeeee`

Text tokens:

Token	Purpose	Dark Value	Light Value
`--color-text-primary`	Body text, headings	`#e8e8e8`	`#1d1d1f`
`--color-text-secondary`	Subdued labels, metadata	`#999999`	`#6e6e73`
`--color-text-tertiary`	Placeholders, disabled text	`#666666`	`#aeaeb2`

Accent tokens:

Token	Purpose	Dark Value	Light Value
`--color-accent`	Primary interactive elements, links	`#5e9eff`	`#0071e3`
`--color-accent-hover`	Hovered interactive elements	`#7bb3ff`	`#0077ed`
`--color-accent-subtle`	Soft accent backgrounds (badges, tags)	`rgba(94,158,255,0.12)`	`rgba(0,113,227,0.08)`

Status tokens:

Token	Purpose	Dark Value	Light Value
`--color-success`	Lint pass, success states	`#30d158`	`#248a3d`
`--color-warning`	Lint warnings	`#ffd60a`	`#b25000`
`--color-error`	Lint errors, validation failures	`#ff453a`	`#d70015`

4.4. Typography

UI typography (toolbars, settings labels, buttons):

Token	Value
`--font-ui`	`"Inter", -apple-system, BlinkMacSystemFont, "Segoe UI", system-ui, sans-serif`
`--font-size-xs`	`11px`
`--font-size-sm`	`12px`
`--font-size-base`	`13px`
`--font-size-lg`	`15px`
`--font-weight-normal`	`400`
`--font-weight-medium`	`500`
`--font-weight-semibold`	`600`

Editor typography (raw markdown source in the CodeMirror pane):

Token	Value
`--font-editor`	`"JetBrains Mono", "Fira Code", "SF Mono", "Cascadia Code", "Consolas", monospace`
`--font-size-editor`	`13px`
`--line-height-editor`	`1.6`

Preview typography (rendered markdown output):

Token	Value
`--font-preview`	`"Inter", -apple-system, BlinkMacSystemFont, "Segoe UI", system-ui, sans-serif`
`--font-preview-mono`	`"JetBrains Mono", "Fira Code", monospace`
`--font-size-preview`	`15px`
`--line-height-preview`	`1.7`

Font preferences for both editor and preview are user-configurable. The values above are compiled defaults. Users may select from a curated list of font stacks (including serif options for preview rendering) or supply a custom font-family string. Font files are not bundled with the application; the system's installed fonts are used.

4.5. Spacing and Layout

All spacing values are multiples of a 4px base unit, following an 8pt-grid-aligned system. This produces a consistent visual rhythm across all components.

Token	Value	Usage
`--space-1`	`4px`	Tight internal padding (icon-to-label gap)
`--space-2`	`8px`	Standard internal padding, small component gaps
`--space-3`	`12px`	Input field padding, list item padding
`--space-4`	`16px`	Section padding, card padding
`--space-6`	`24px`	Panel margins, large section gaps
`--space-8`	`32px`	Page-level margins
`--radius-sm`	`4px`	Buttons, inputs, small cards
`--radius-md`	`8px`	Panels, modals, dropdown menus
`--radius-lg`	`12px`	Application window corners (where supported)

4.6. Component Styling

Toolbar. A single horizontal bar at the top of the viewport. Height: `40px`. Background: `--color-bg-secondary`. Bottom border: `1px solid --color-border-primary`. Contains the view-mode toggle, file name display, and action buttons. Buttons use icon-only presentation with tooltips; no text labels in the toolbar.

Buttons. Two visual variants:

- *Ghost* (default): transparent background, `--color-text-secondary` foreground. On hover: `--color-bg-hover` background, `--color-text-primary` foreground.
- *Accent*: `--color-accent` background, white foreground. On hover: `--color-accent-hover` background. Used sparingly (primary conversion actions, confirm dialogs).

All buttons use `--radius-sm` border radius. Minimum tap target: `32px` square. Focus states use a `2px` outline offset with `--color-accent`.

Inputs. Background: `--color-bg-tertiary`. Border: `1px solid --color-border-primary`. On focus: border transitions to `--color-accent`. Padding: `--space-2` horizontal, `--space-1` vertical. Text: `--font-size-base`.

Panels and dividers. Pane dividers (e.g., the split-view separator) are `1px solid --color-border-primary`. The divider is draggable in split-view to resize pane proportions. Drag affordance: on hover, the divider thickens to `3px` and changes color to `--color-accent-subtle`.

Tooltips. Background: `--color-bg-active`. Text: `--color-text-primary`. Font size: `--font-size-xs`. Border radius: `--radius-sm`. Padding: `--space-1` vertical, `--space-2` horizontal. Appear after a 400ms hover delay, fade in over 120ms.

Modals. Centered overlay with a backdrop of `rgba(0,0,0,0.5)`. Modal surface: `--color-bg-secondary`, `--radius-md` border radius, `1px solid --color-border-primary`. Max width: `480px`. Content padding: `--space-6`. Entrance animation: fade in + scale from 0.97 to 1.0 over 150ms.

4.7. Motion and Transitions

Motion is functional, not decorative. Transitions exist to maintain spatial continuity (e.g., pane resizing) and to signal state changes (e.g., theme switching). No animation should draw attention to itself.

Context	Duration	Easing	Property
Hover state changes	`120ms`	`ease-out`	`background-color`, `color`, `border-color`
View mode toggle	`200ms`	`ease-in-out`	`width`, `opacity`
Theme switch	`250ms`	`ease-in-out`	All color tokens via CSS transition on `[data-theme]`
Modal entrance	`150ms`	`cubic-bezier(0.16, 1, 0.3, 1)`	`opacity`, `transform`
Tooltip appearance	`120ms`	`ease-out`	`opacity`
Pane resize (drag)	`0ms`	—	`width` (immediate, no transition during active drag)

Users who enable `prefers-reduced-motion` at the OS level receive instant state changes with no transitions. The application MUST respect this preference.

4.8. Iconography

Icons use the Lucide icon set (the fork of Feather Icons). All icons render at `16px` with a `1.5px` stroke weight for the default toolbar size. Icons inherit the current text color via `currentColor`. No filled icon variants are used; the entire icon language is outlined/stroked for visual consistency.

---

5. Application Layout

5.1. View Modes

The application has four top-level views, accessed via the toolbar or keyboard shortcuts:

View	Shortcut	Description
Full-view	`Ctrl/Cmd+1`	Rendered markdown preview only. Default on file open.
Split-view	`Ctrl/Cmd+2`	Side-by-side editor and preview.
Library	`Ctrl/Cmd+3`	File management table. Desktop only.
Settings	`Ctrl/Cmd+,`	Configuration panel.

5.2. Full-View Mode

Full-view is the default. The rendered markdown fills the viewport below the toolbar. Content is horizontally centered with a maximum reading width of `720px` and horizontal padding of `--space-8` on each side. This produces a comfortable, Notion-like reading column that prevents excessively long line lengths on wide displays.

When viewing a local file on desktop, the application watches the file for changes on disk. If the source file is modified externally, the preview re-renders automatically with no user action required. A subtle flash indicator (a brief `200ms` pulse of `--color-accent-subtle` on the document background) signals that the content has refreshed.

5.3. Split-View Mode

Split-view places the CodeMirror editor on the left and the rendered preview on the right. The default split ratio is 50/50. A draggable divider allows the user to resize the panes. The minimum width for either pane is `280px`. Pane proportions are persisted in configuration and restored on next launch.

Scroll synchronization: the preview pane scroll position tracks the editor cursor position. When the cursor moves in the editor, the preview scrolls to bring the corresponding rendered content into view. This synchronization uses a proportional mapping (editor scroll percentage maps to preview scroll percentage) rather than per-line tracking, which avoids visual jitter with content that expands during rendering (tables, code blocks, images).

5.4. Library View

The library view replaces the editor/preview area with a sortable, filterable data table. See §9 for full library behavior. The toolbar adapts to show library-specific controls (mount directory button, refresh, filter input).

5.5. Settings View

Settings are presented as a single scrollable panel organized into labeled sections. Each section is a card-like region with a section heading, a brief description, and the associated controls. The visual treatment follows the Apple System Settings / Linear Settings pattern: clean vertical stacking with generous whitespace between sections and no tab navigation within settings.

Settings sections (in order of appearance):

1. **Appearance** — Color mode toggle (Light / Dark / System), visual style selection.
2. **Editor** — Font selection, font size, linting toggle, linter selection.
3. **Preview** — Font selection, font size, line height.
4. **Markdown Engine** — Engine selection dropdown.
5. **File Extensions** — Extension whitelist editor.
6. **Library** (desktop only) — Mount directory, recursion toggle, hidden file toggle, independent extension rules toggle.

---

6. Core Features

6.1. Viewing

Local files. The application opens local markdown files either via CLI invocation (`shruggie-md <filepath>`), OS file association (double-click), or a file-open dialog within the application. The file is read from disk, compiled through the selected markdown engine, and rendered in the preview pane.

Live reload. On desktop, a filesystem watcher (Tauri's `watch` API) monitors the open file. When the file's modification timestamp changes, the source is re-read and the preview re-renders. The watcher debounces rapid successive changes with a 300ms delay to avoid unnecessary render cycles during active external editing.

Remote files. The application can fetch and render raw markdown content from a public URL. Remote fetching is invoked through the toolbar (a URL input field) or via CLI (`shruggie-md --url <url>`). The application sends a plain `GET` request with no authentication headers. If the response content-type is not a text type, the application displays an error. A small info badge (`ⓘ`) in the toolbar opens a modal with usage instructions for remote fetching.

6.2. Editing

Editing is available in split-view mode. The CodeMirror editor provides the editing surface with full markdown syntax highlighting, configurable linting, and standard editor conveniences (undo/redo, find/replace, line numbers). Changes in the editor are reflected in the preview pane in real-time with a 150ms debounce on the render cycle.

On desktop, edits can be saved back to the source file via `Ctrl/Cmd+S`. The save operation writes the editor content to disk atomically (write to temp file, then rename). In the Chrome extension and PWA targets, the save operation downloads the file to the user's default download location.

6.3. Converting

Markdown to HTML. The compiled HTML output (the same output rendered in the preview pane) is available for export via `Ctrl/Cmd+Shift+H` or the toolbar. The exported HTML is a self-contained file with all styles inlined. No external stylesheet references are included.

Markdown to PDF. PDF export uses the browser's native print pathway:

1. The markdown is compiled to HTML.
2. A print-specific stylesheet is applied. This stylesheet sets standard print margins, page numbers in the footer, no URL footers, and appropriate page-break behavior.
3. `<hr>` elements in the source markdown are automatically replaced with CSS `page-break-before: always` rules, producing clean page breaks at horizontal rule positions.
4. The print dialog is invoked (or, on desktop via Tauri, a direct PDF write operation if available).

PDF export is triggered via `Ctrl/Cmd+Shift+P` or the toolbar.

6.4. Remote URL Fetching

Remote URL fetching supports only publicly accessible endpoints. The application sends a plain HTTP `GET` request. CORS restrictions apply when running in browser contexts (Chrome extension, PWA). The Chrome extension uses the `host_permissions` declaration in the manifest to bypass CORS for user-specified URLs.

Fetched content is treated as read-only. The editor is available in split-view for inspection, but save operations are disabled for remote content. Fetched content is not cached to disk.

---

7. Editor Engine

7.1. CodeMirror Integration

The editor pane is a CodeMirror 6 instance configured for markdown editing. CodeMirror is chosen for its modular architecture, excellent TypeScript support, accessibility compliance, and extensibility for syntax highlighting and linting integration.

Required CodeMirror extensions:

Extension	Purpose
`@codemirror/lang-markdown`	Markdown language mode with syntax tree
`@codemirror/language-data`	Language detection for fenced code blocks
`@codemirror/search`	Find and replace
`@codemirror/autocomplete`	Optional completion hints
`@codemirror/lint`	Lint integration framework
`@codemirror/view` (line numbers, highlight active line)	Core editor features
`@codemirror/state`	Editor state management

The editor theme is derived from the application's color tokens. A custom CodeMirror theme is constructed at runtime from the active CSS custom property values, ensuring the editor appearance always matches the selected application theme.

7.2. Syntax Highlighting

Markdown syntax highlighting in the editor MUST clearly distinguish:

Element	Styling Approach
Headings (`#` , `##` , etc.)	Bold, scaled font size, `--color-text-primary`
Bold / italic markers	Dimmed marker characters (`--color-text-tertiary`), styled content text
Links	URL portion in `--color-text-secondary` , link text in `--color-accent`
Code spans and fenced blocks	`--font-editor` (monospace), `--color-bg-hover` background
Blockquotes	Left border accent, slightly dimmed text
List markers	`--color-accent` for bullet/number characters
Horizontal rules	`--color-border-primary` styled line

Fenced code blocks receive language-specific syntax highlighting via `@codemirror/language-data` , which provides grammar definitions for common programming languages. The highlighting inside fenced code blocks uses the same color palette as

the surrounding markdown but with language-aware token coloring.

7.3. Markdown Linting

Linting is optional and disabled by default. When enabled, lint diagnostics appear as:

- Inline underlines (wavy for errors, dotted for warnings) beneath the offending text.
- A tooltip popup on hover showing the rule ID, severity, and a human-readable description of the issue.
- A summary count (errors/warnings) in the toolbar's status area.

Available linters:

Linter	Package	Description
markdownlint	`markdownlint` (via `markdownlint-rule-helpers`)	The widely adopted standard. Strong default rule set covering heading style, list consistency, line length, and more.
remark-lint	`remark-lint` (via the unified/remark ecosystem)	Plugin-based linter with fine-grained rule composition. Integrates naturally with the remark markdown engine if selected.

Users may switch between linters in the Editor section of settings. Both linters are bundled; switching is instant and does not require a reload. The active linter's diagnostics are piped into CodeMirror's lint extension via a custom lint source function that adapts each linter's output format to the CodeMirror `Diagnostic` interface.

---

8. Markdown Rendering

8.1. Rendering Pipeline

The rendering pipeline transforms markdown source into styled HTML displayed in the preview pane:

1. Raw markdown source (from file, editor buffer, or remote fetch).
2. Markdown engine compiles source to an HTML string.
3. HTML is sanitized (DOMPurify) to prevent XSS from untrusted content.
4. Sanitized HTML is injected into the preview container.
5. Post-render hooks apply syntax highlighting to code blocks (via highlight.js or Shiki, configurable).

8.2. Selectable Engines

The markdown engine is selectable in the technical options section of settings. This is an advanced feature; the default engine (markdown-it) is suitable for the vast majority of users. Engine selection is exposed to support power users who have specific rendering requirements or preferences.

Engine	Package	Default	Notes
markdown-it	`markdown-it`	Yes	Fast, CommonMark-compliant, rich plugin ecosystem. Supports GFM tables, task lists, footnotes via plugins.
marked	`marked`	No	Lightweight and fast. Fewer extension points than markdown-it.
remark (unified)	`remark` + `remark-html`	No	AST-based pipeline. Maximum extensibility. Pairs naturally with remark-lint.

All engines MUST produce output that passes through the same sanitization and styling pipeline. Engine selection affects parsing behavior (e.g., how edge-case markdown constructs are interpreted), not the visual presentation of the rendered output.

8.3. Rendered Content Styling

The preview pane applies a dedicated stylesheet to rendered HTML. This stylesheet is independent of the editor theme and is designed for comfortable reading.

Key rendered content styles:

- **Headings.** `h1` through `h6` use `--font-preview` with descending sizes (28px, 22px, 18px, 16px, 15px, 13px). Bottom border on `h1` and `h2` (`1px solid --color-border-subtle`). Margin top: `1.5em` , margin bottom: `0.5em` .
- **Paragraphs.** `--font-size-preview` , `--line-height-preview` . Margin bottom: `1em` .
- **Code blocks.** Background: `--color-bg-tertiary` . Padding: `--space-4` . Border radius: `--radius-sm` . Font: `--font-preview-mono` . Overflow: horizontal scroll.
- **Inline code.** Background: `--color-bg-hover` . Padding: `2px 6px` . Border radius: `3px` .
- **Blockquotes.** Left border: `3px solid --color-accent-subtle` . Padding left: `--space-4` . Text color: `--color-text-secondary` .

- **Tables.** Full width. Header row: `--color-bg-secondary` background, `--font-weight-semibold`. Cell padding: `--space-2` vertical, `--space-3` horizontal. Borders: `1px solid --color-border-subtle`.
- **Links.** Color: `--color-accent`. No underline by default; underline on hover.
- **Images.** Max width: `100%`. Border radius: `--radius-sm`. Centered within the reading column.
- **Horizontal rules.** `1px solid --color-border-subtle`. Margin: `--space-6` vertical.
- **Task lists.** Custom checkbox styling matching the application's visual language (accent color when checked, subtle border when unchecked).

---

9. Library Mode

9.1. Purpose

Library mode provides a flat file management interface for users who maintain a collection of markdown files in a single directory tree. It is available only in the native desktop application (Tauri), not in the Chrome extension or PWA, because it requires direct filesystem access to a user-designated directory.

9.2. Setup

The user configures a single directory as the library root via the Library section of settings. This directory is "mounted" as the library. Only one directory may be mounted at a time. Changing the mounted directory replaces the previous library contents entirely; there is no merge behavior.

9.3. Library Table

The library view displays a sortable data table with the following columns:

Column	Content	Sort Behavior
Title	Filename without extension	Alphabetical (case-insensitive)
Path	Absolute path to the file	Alphabetical
Last Edited	ISO 8601 date-time with UTC offset (`mtime`)	Chronological (newest first default)
Created	ISO 8601 date-time with UTC offset (`birthtime`)	Chronological

Clicking a row opens the file in full-view mode. The table supports multi-column sorting (click a column header to sort; shift-click to add a secondary sort). A text filter input in the toolbar performs substring matching against the Title and Path columns.

9.4. Traversal and Filtering

- By default, the library recursively traverses all subdirectories within the mounted root.
- A toggle in Library settings disables recursion, limiting the library to top-level files only.
- A separate toggle controls whether hidden files and directories (names starting with `.`) are included. Default: hidden items are excluded.
- Non-markdown files (as determined by the active extension rules) are silently ignored. They do not appear in the table and no error is produced.

9.5. Extension Rules

By default, library mode follows the same file extension rules used in the application's global file extension settings (§10.4). A toggle in the Library settings section allows library mode to define its own independent set of recognized extensions. When this toggle is enabled, a separate extension list editor appears within the Library settings.

10. Configuration and Preferences

10.1. Configuration Architecture

Configuration is stored as JSON. On desktop (Tauri), preferences are read from and written to the platform-standard application data directory (§10.5). In the Chrome extension, preferences use the `chrome.storage.sync` API. In the PWA, preferences use `IndexedDB` via a thin wrapper.

All configuration keys have compiled defaults in the application source. User-set values override defaults. Unknown keys in persisted configuration are silently ignored for forward compatibility.

10.2. Theme and Appearance

Setting	Type	Default	Options
Color mode	enum	`dark`	`light`, `dark`, `system`
Visual style	enum	`default`	A small curated set of named styles (e.g., `default`, `warm`, `cool`, `monochrome`)

Visual styles adjust the color token values within the active color mode. For example, the `warm` style shifts surface colors toward amber tones, while the `monochrome` style reduces the accent palette to grayscale. The style system is implemented as named sets of CSS custom property overrides layered on top of the base light/dark tokens.

10.3. Editor Preferences

Setting	Type	Default
Font family	string (font stack)	`"JetBrains Mono", "Fira Code", monospace`
Font size	number (px)	`13`
Line height	number	`1.6`
Show line numbers	boolean	`true`
Word wrap	boolean	`true`
Linting enabled	boolean	`false`
Active linter	enum	`markdownlint`

10.4. File Extension Rules

The application maintains a configurable set of file extensions it recognizes as markdown. The default set is: `.md`, `.markdown`, `.mdown`, `.mkdn`, `.mkd`. Files that do not match any extension in the set are rejected with a user-visible error message suggesting that the user modify the recognized extensions in settings.

These rules apply in file viewing mode and (by default) in library mode. Library mode may optionally override these rules with its own independent set (§9.5).

10.5. Storage Locations

Platform	Path
Windows	`%LOCALAPPDATA%\shruggie-md\`
Linux	`$XDG_CONFIG_HOME/shruggie-md/` (default: `~/.config/shruggie-md/`)
macOS	`~/Library/Application Support/shruggie-md/`
Chrome extension	`chrome.storage.sync`
PWA	IndexedDB (`shruggie-md-config` store)

---

11. Platform Targets

11.1. Desktop Application

All desktop targets share the following requirements:

- The system PATH includes a CLI-invocable `shruggie-md` command that launches the application.
- Preferences are stored in the platform-standard application data location (§10.5).
- Library mode is available.
- File watching (live reload) is available.

Windows 11 (primary target):

- Installer offers a desktop icon.
- Installer prompts the user to set the application as the default handler for `.md` and `.markdown` files.
- MSI-based installer via Tauri's WiX integration.

macOS (arm64):

- DMG distribution with standard drag-to-Applications install.
- Application registers as a handler for `.md` and `.markdown` file types via `Info.plist`.

Linux (x64):

- AppImage distribution for maximum compatibility across distributions.
- `.desktop` file included for desktop environment integration.

11.2. Chrome Extension

The Chrome extension renders markdown files opened in the browser. When the user navigates to a URL ending in a recognized markdown extension, the extension intercepts the raw content and renders it using the Shruggie Markdown preview. The extension also provides a browser action popup for pasting or typing a URL to fetch and render.

Manifest V3 configuration:

- `content_scripts`: match patterns for recognized markdown file extensions.
- `host_permissions`: user-configurable URL patterns for remote fetch (to bypass CORS).
- `action`: browser action popup for URL input and settings access.
- `storage`: `chrome.storage.sync` for configuration persistence.

The Chrome extension does NOT support library mode or file watching. These features require filesystem access unavailable in the extension context.

11.3. Progressive Web App

The PWA is a hosted static site that serves the Shruggie Markdown frontend with a service worker for offline capability and a web app manifest for installability. The PWA supports markdown viewing, editing, and conversion. It does not support library mode or file watching.

PWA configuration:

- Service worker (Workbox): precaches the application shell and core assets. Runtime caching for fetched remote markdown content.
- Web app manifest: `name`, `short_name`, `icons`, `start_url`, `display: standalone`, `theme_color` matching the dark mode primary background.
- Install prompt: a subtle, non-intrusive banner on first visit offering installation.

---

12. Repository Structure

```
shruggie-md/
├── src/
│   ├── components/          # React UI components
│   │   ├── Editor/          # CodeMirror editor pane
│   │   ├── Preview/         # Markdown preview pane
│   │   ├── Library/         # Library table view
│   │   ├── Settings/        # Settings panel
│   │   ├── Toolbar/         # Top toolbar
│   │   └── common/          # Shared UI primitives (Button, Input, Modal, Tooltip)
│   ├── engines/             # Markdown engine abstraction and implementations
│   ├── linters/             # Linter adapter layer
│   ├── platform/            # Platform abstraction (filesystem, storage, capabilities)
│   ├── styles/              # Global CSS, design tokens, print stylesheet
│   ├── hooks/               # React hooks (useConfig, useTheme, useFileWatcher, etc.)
│   ├── types/               # TypeScript type definitions
│   ├── App.tsx              # Root application component
│   └── main.tsx             # Entry point
├── src-tauri/               # Tauri backend (Rust)
│   ├── src/
│   │   └── main.rs          # Tauri application entry, CLI registration, file watcher
│   ├── Cargo.toml
│   ├── tauri.conf.json
│   └── icons/
├── extension/               # Chrome extension manifest and content scripts
│   ├── manifest.json
│   ├── content.ts
│   └── popup.html
├── public/                  # Static assets, PWA manifest, service worker registration
│   ├── manifest.json
│   └── icons/
├── index.html               # Vite entry HTML
├── vite.config.ts
├── tailwind.config.ts
├── tsconfig.json
├── package.json
├── LICENSE
├── README.md
└── shruggie-md-spec.md
```

13. Future Considerations

The following capabilities are deferred from the MVP and may be pursued in future releases:

- **Authenticated remote URL fetching.** Support for bearer tokens, API keys, or basic auth when fetching markdown from private endpoints.
- **Chrome OS targeting.** Investigation into whether the Chrome extension works natively on Chrome OS or requires a separate build.
- **Additional visual styles.** Expanding the curated style set beyond the initial four.
- **Custom themes.** A theme editor allowing users to define their own color token values.
- **Plugin system.** An extension mechanism for custom markdown-it plugins, custom linter rules, or post-processing hooks.
- **Export to additional formats.** DOCX, EPUB, or LaTeX export pipelines.
- **Multiple library roots.** Supporting more than one mounted directory simultaneously.
- **File tagging and search.** Full-text search across library contents, with optional tag/label metadata.
- **Vim/Emacs keybindings.** Alternative keybinding modes in the CodeMirror editor.

Document History

Date	Version	Change
2026-03-24	DRAFT	Initial specification. Derived from the product brainstorm document (`20260324-001-Brainstorm.md`). Establishes architecture, visual design language, feature set, and platform targets for MVP development.